In [1]:
```
#Hemant Marotkar ,UID: -23015002

# Experiment: Extract features from phishing dataset and apply SVM classification
# Dataset file required: phishing_dataset.csv

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load the phishing dataset
# Each row represents a website/URL and extracted phishing-related features.
df = pd.read_csv("phishing_dataset.csv")

print("First five records of dataset:")
print(df.head())
print("\nDataset information:")
print(df.info())

# 2. Separate input features and output label
# X contains feature columns; y contains class label: phishing or legitimate.
X = df.drop("label", axis=1)
y = df["label"]

# 3. Convert text labels into numeric labels
# legitimate/phishing are converted into 0/1 values for machine learning.
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

print("\nLabel mapping:")
for class_name, encoded_value in zip(label_encoder.classes_, range(len(label_encode
    print(class_name, "=", encoded_value)

# 4. Split dataset into training and testing data
# 70% data is used for training and 30% for testing.
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.30,
    random_state=42,
    stratify=y_encoded
)

# 5. Feature scaling
# SVM is distance-based, so all features should be on a similar scale.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Create and train SVM classifier
# RBF kernel helps classify non-linear patterns in phishing data.
svm_model = SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)
svm_model.fit(X_train_scaled, y_train)
```

```
# 7. Predict labels for test data
y_pred = svm_model.predict(X_test_scaled)

# 8. Evaluate model performance
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=label_encoder.classes_)

print("\nModel Evaluation Results")
print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(report)

# 9. Test the model using a new sample URL feature record
# Feature order must be same as CSV columns except label.
feature_columns = X.columns

new_sample = pd.DataFrame(
    [[90, 1, 1, 1, 1, 5, 1, 1, 4, 1, 7, 1000000]],
    columns=feature_columns
)
new_sample_scaled = scaler.transform(new_sample)
predicted_class = svm_model.predict(new_sample_scaled)
predicted_label = label_encoder.inverse_transform(predicted_class)

print("\nPrediction for new sample website:", predicted_label[0])
```

```
First five records of dataset:
   url_length  has_ip_address  has_at_symbol  has_hyphen  prefix_suffix  \
0          75               1              0           1              1
1          54               0              0           0              0
2          92               1              1           1              1
3          43               0              0           0              0
4          81               0              1           1              1

   dots_count  https_token  shortening_service  subdomain_count  \
0           4            1                   0                3
1           2            0                   0                1
2           5            1                   1                4
3           1            0                   0                0
4           4            0                   1                3

   has_suspicious_words  domain_age_days  web_traffic_rank       label
0                     1               20            950000    phishing
1                     0             1500             12000  legitimate
2                     1                5           1200000    phishing
3                     0             2500              8500  legitimate
4                     1               12            780000    phishing

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   url_length            30 non-null     int64
 1   has_ip_address        30 non-null     int64
 2   has_at_symbol         30 non-null     int64
 3   has_hyphen            30 non-null     int64
 4   prefix_suffix         30 non-null     int64
 5   dots_count            30 non-null     int64
 6   https_token           30 non-null     int64
 7   shortening_service    30 non-null     int64
 8   subdomain_count       30 non-null     int64
 9   has_suspicious_words  30 non-null     int64
 10  domain_age_days       30 non-null     int64
 11  web_traffic_rank      30 non-null     int64
 12  label                 30 non-null     str
dtypes: int64(12), str(1)
memory usage: 3.4 KB
None

Label mapping:
legitimate = 0
phishing = 1

Model Evaluation Results
Accuracy: 100.0 %

Confusion Matrix:
[[5 0]
 [0 4]]
```

```
Classification Report:
              precision    recall  f1-score   support

  legitimate       1.00      1.00      1.00         5
    phishing       1.00      1.00      1.00         4

    accuracy                           1.00         9
   macro avg       1.00      1.00      1.00         9
weighted avg       1.00      1.00      1.00         9


Prediction for new sample website: phishing
```